

FASE 0 · O AGENTE EM CÓDIGO PURO

Aula 0.4

Registro de ferramentas e schema portátil

Um decorator lê a sua função e escreve o schema nos dois dialetos sozinho. Adicionar uma ferramenta vira escrever uma função.

Série: Engenharia de IA Aplicada · Agentes e RAG em Produção

Projeto: Núcleo de Agentes Operacionais

Duração de estudo: cerca de 1h30 · 25 min de teoria e 55 de código

Pré-requisito: Aulas 0.1 a 0.3 (tag **aula-0.3**)

1. Objetivo de aprendizagem

Ao final desta aula você será capaz de registrar uma ferramenta apenas escrevendo uma função Python tipada e documentada, com o schema de cada provider gerado automaticamente por introspecção, sem tocar no laço da aula anterior.

2. Por que esta aula importa

Na aula 0.3 você escreveu cada ferramenta três vezes: a função Python, o schema no dialeto da Anthropic e o schema no dialeto da OpenAI. Com duas ferramentas isso incomoda. Com as quinze de um sistema real, vira fonte de bug silencioso.

O cenário é concreto e comum: alguém renomeia um parâmetro de `data` para `data_inicio`, uma melhoria legítima, e atualiza o schema do provider que a equipe usa no dia a dia. O outro fica para trás. Nada quebra: o sistema segue de pé, os testes verdes. Só que todo cliente rodando no segundo provider passa a ter um agente chamando a ferramenta com um argumento que já não existe, recebendo erro e contornando como pode. A qualidade cai em silêncio, e silêncio, em produção, é o adjetivo mais caro que existe.

A solução tem nome em engenharia: **fonte única da verdade**. A função Python já contém tudo que o schema precisa dizer, e a docstring já contém a descrição. Então o schema não deve ser escrito. Deve ser **derivado**.

3. Teoria

3.1 Definição e execução são coisas diferentes

Uma ferramenta tem duas metades que hoje estão coladas no seu código. A **definição** é o contrato que o modelo enxerga: nome, descrição, parâmetros, tipos. É texto, e é serializado dentro da chamada de API. A **execução** é a função Python que roda quando o modelo pede. É código, e roda na sua máquina.

O modelo nunca vê a execução. Sua função nunca vê o modelo. O que liga as duas metades é apenas o **nome**. É esse desacoplamento que permite trocar de framework depois sem reescrever o domínio, e é ele que o MCP formaliza na Fase 2.

3.2 Introspecção: a informação já está na função

O módulo `inspect`, da biblioteca padrão, entrega tudo:

- `inspect.signature(funcao)` devolve cada parâmetro, seu type hint e se tem valor padrão.
- `inspect.getdoc(funcao)` devolve a docstring já limpa de indentação.
- `funcao.__name__` devolve o nome.

Ou seja: se você escreve uma função bem tipada e bem documentada, que é o que já deveria fazer, toda a informação do schema já está lá. O decorator só coleta e traduz. Vale um minuto no REPL para ver ao vivo:

```
>>> import inspect
>>> def exemplo(email: str, limite: int = 5): ...
>>> for nome, p in inspect.signature(exemplo).parameters.items():
...     print(nome, p.annotation, p.default)
```

```
email <class 'str'> <class 'inspect._empty'>
limite <class 'int'> 5
```

Repare: o parâmetro sem valor padrão aparece como `inspect._empty`. É exatamente esse sinal que usamos para decidir o que é obrigatório.

3.3 O schema neutro e os dois dialetos

Conceito	Anthropic	OpenAI
Nome	<code>name</code> no topo	<code>function.name</code>
Descrição	<code>description</code> no topo	<code>function.description</code>
Parâmetros	<code>input_schema</code>	<code>function.parameters</code>
Envelope	nenhum	<code>{"type": "function", ...}</code>

O miolo dos parâmetros é **idêntico**: JSON Schema puro. Isso não é coincidência. JSON Schema é o padrão que a indústria adotou para descrever ferramentas, inclusive o MCP da Fase 2. A estratégia, então: o registro guarda um **formato neutro** e duas funções pequenas geram cada dialeto na hora. Mesmo padrão do `mensagens_de_resultado` da aula 0.3: a diferença entre providers vive num ponto só do código.

3.4 O decorator como cartório

Um decorator em Python é só uma função que recebe uma função e devolve algo. Normalmente devolve uma versão modificada. O nosso faz diferente: ele **não modifica nada**. Lê a função, monta a definição neutra, guarda no registro e devolve a função original intocada.

ANALOGIA

O decorator aqui é um carimbo de cartório: registra a existência da ferramenta e vai embora. A função continua sendo Python comum, chamável direto nos testes, sem passar pelo agente. Isso importa mais do que parece: ferramenta que só roda dentro do agente é ferramenta difícil de testar.

3.5 Falhar no registro, não em produção

Uma decisão de projeto que vale discutir: o decorator **levanta exceção** se faltar type hint ou docstring. Isso quebra a importação do módulo, na hora, no seu terminal.

É deliberado. O alternativo seria gerar um schema incompleto e descobrir o problema em produção, quando o modelo errar o argumento ou simplesmente ignorar a ferramenta. Entre falhar alto no desenvolvimento e falhar em silêncio na frente do cliente, a escolha é fácil. Contrato incompleto é erro de programação, não condição de runtime.

3.6 O que fica portátil

Guarde esta frase: **o contrato da ferramenta é portátil; a embalagem não é**. Nome, descrição, tipos e obrigatórios sobrevivem à troca de provider, de framework e até de protocolo. O que muda é só a serialização final. Centralizar o contrato num formato neutro garante que trocar de stack nas Fases 1 e 2 seja tradução, não reescrita.

4. Mão na massa

Esta aula cria **um arquivo novo**, `nucleo/registro.py`, e seus testes. Nenhum arquivo anterior é alterado. Em especial, o `agente.py` da aula 0.3 não muda uma linha, e isso é a prova de que o desacoplamento funcionou.

Passo 1 — O esqueleto do registro

O que e por quê: a estrutura de dados neutra e o dicionário central. Um `dataclass` para o contrato, um `dict` como cartório.

```
"""Aula 0.4 - Registro de ferramentas e schema portavel.

Uma ferramenta tem duas metades: a DEFINICAO (o contrato que o modelo
enxerga) e a EXECUCAO (a funcao Python que roda). Este modulo guarda as
duas num lugar so e gera o schema de cada provider automaticamente.

A regra da aula: adicionar uma ferramenta nova e escrever uma funcao.
Nada mais. O laco da Aula 0.3 nao muda uma linha.
"""

import inspect                                # le assinatura e docstring das funcoes
from dataclasses import dataclass, field

# Traducao de type hint do Python para tipo do JSON Schema. Sao os
# quatro tipos simples que cobrem a maioria das ferramentas reais.
# Tipos compostos ficam para quando o projeto precisar: nao
# antecipamos complexidade.
MAPA_TIPOS = {
    str: "string",
    int: "integer",
    float: "number",
    bool: "boolean",
}

@dataclass
class Ferramenta:
    """0 contrato neutro de uma ferramenta, sem dialeto de provider."""
    nome: str                # liga a definicao a execucao
    descricao: str           # o que o modelo le para decidir usar
    funcao: object           # a funcao Python que executa de verdade
    propriedades: dict = field(default_factory=dict) # JSON Schema dos params
    obrigatorios: list = field(default_factory=list) # params sem default

# 0 cartorio: nome da ferramenta -> Ferramenta. E um dict de modulo,
# entao importar nucleo.registro em qualquer lugar enxerga o mesmo
# registro.
REGISTRO = {}
```

O que faz: `MAPA_TIPOS` traduz type hint para tipo do JSON Schema. Quatro tipos simples bastam para a maioria das ferramentas reais; tipos compostos entram quando o projeto precisar. `REGISTRO` é um dict de módulo, então qualquer lugar que importe `nucleo.registro` enxerga o mesmo cartório.

Passo 2 — O decorator

O que e por quê: o coração da aula. Lê a assinatura, decide o que é obrigatório, valida o contrato e registra.

```
def ferramenta(func):
    """Decorator que REGISTRA a funcao como ferramenta do agente.

    Ele nao modifica nada: le a assinatura e a docstring, monta o
    contrato neutro, guarda no REGISTRO e devolve a funcao ORIGINAL,
    intocada. Por isso voce continua podendo chamar a funcao como
    Python comum, inclusive nos testes.
    """
    assinatura = inspect.signature(func)

    propriedades = {}
    obrigatorios = []

    for nome_param, param in assinatura.parameters.items():
        anotacao = param.annotation

        if anotacao is inspect.Parameter.empty:
            # Sem type hint nao da para gerar schema. Falhar alto aqui
            # e melhor do que o modelo receber um contrato incompleto
            # e errar o argumento em producao.
            raise TypeError(
                f"A ferramenta '{func.__name__}' tem o parametro "
                f"'{nome_param}' sem type hint. Anote o tipo."
            )

        tipo_json = MAPA_TIPOS.get(anotacao)
        if tipo_json is None:
            raise TypeError(
                f"Tipo nao suportado em '{func.__name__}.{nome_param}': "
                f"{anotacao}. Use str, int, float ou bool."
            )

        propriedades[nome_param] = {"type": tipo_json}

        # Sem valor padrao = obrigatorio. E o sinal que o inspect da.
        if param.default is inspect.Parameter.empty:
            obrigatorios.append(nome_param)

    descricao = inspect.getdoc(func) or ""
    if not descricao:
        # Descricao de ferramenta e PROMPT: e o texto que o modelo le
        # para decidir usar. Sem ela, a acuracia de selecao despenca.
        raise ValueError(
            f"A ferramenta '{func.__name__}' nao tem docstring. "
            "A descricao e o que o modelo le para decidir usa-la."
        )

    REGISTRO[func.__name__] = Ferramenta(
        nome=func.__name__,
        descricao=descricao,
        funcao=func,
        propriedades=propriedades,
        obrigatorios=obrigatorios,
    )

    return func # a funcao ORIGINAL, sem wrapper

# -----
```

```
# TRADUCAO: o miolo (JSON Schema) e identico; so a embalagem muda.  
# -----
```

O que faz: percorre os parâmetros, traduz cada tipo, marca como obrigatório o que não tem valor padrão, e exige docstring. Note o `return func` no final: a função sai daqui exatamente como entrou.

Passo 3 — A tradução para cada dialeto

O que e por quê: três funções pequenas. Uma monta o miolo, comum aos dois; duas montam a embalagem de cada provider. Mais duas de conveniência entregam exatamente o formato que `rodar_agente` já espera desde a aula 0.3.

```
def _miolo(f):  
    """O JSON Schema dos parametros, igual nos dois providers."""  
    return {  
        "type": "object",  
        "properties": f.propriedades,  
        "required": f.obrigatorios,  
    }  
  
def _schema_anthropic(f):  
    # Anthropic: name, description e input_schema no topo, sem envelope.  
    return {"name": f.nome, "description": f.descricao,  
            "input_schema": _miolo(f)}  
  
def _schema_openai(f):  
    # OpenAI: tudo dentro de "function"; o schema chama "parameters".  
    return {"type": "function", "function": {  
        "name": f.nome, "description": f.descricao,  
        "parameters": _miolo(f)}}  
  
def schemas_para(provider):  
    """Lista de schemas de TODAS as ferramentas, no dialeto pedido."""  
    if provider == "anthropic":  
        return [_schema_anthropic(f) for f in REGISTRO.values()]  
    if provider == "openai":  
        return [_schema_openai(f) for f in REGISTRO.values()]  
    raise ValueError(f"Provider desconhecido: {provider}")  
  
def schemas():  
    """O dict {"anthropic": [...], "openai": [...]} que rodar_agente  
    espera. Substitui os schemas escritos a mao na Aula 0.3."""  
    return {"anthropic": schemas_para("anthropic"),  
            "openai": schemas_para("openai")}  
  
def executores():  
    """O dict {nome: funcao} que rodar_agente espera."""  
    return {nome: f.funcao for nome, f in REGISTRO.items()}  
  
def executar_ferramenta(nome, argumentos):  
    """Executa a ferramenta pelo nome, com os argumentos do modelo.  
  
    A ligacao entre o que o modelo pediu e o que roda e apenas o NOME.  
    Esse desacoplamento e o que permite trocar de framework depois sem  
    reescrever o dominio, e e o que o MCP formaliza na Fase 2.
```

```

"""
f = REGISTRO.get(nome)
if f is None:
    # Nao levantamos excecao: ferramenta desconhecida vira
    # observacao que o agente le e contorna (Aulas 0.3 e 0.6).
    return {"erro": f"ferramenta desconhecida: {nome}"}
return f.funcao(**argumentos)

```

O que faz: `schemas()` e `executores()` existem para que o laço da aula anterior seja alimentado sem nenhuma adaptação. Era o contrato implícito que a 0.3 deixou pronto.

Passo 4 — Ver funcionando

O que e por quê: as mesmas duas ferramentas da aula 0.3, agora sem uma única linha de schema escrita à mão. Rascunho em `scratch/`, que o `.gitignore` ignora.

```

# scratch/rascunho_0_4.py (descartavel: o .gitignore ignora scratch/)
from nucleo.registro import ferramenta, schemas, executores
from nucleo.agente import rodar_agente

@ferramenta
def buscar_info_lead(email: str) -> dict:
    """Busca empresa, plano e MRR de um lead pelo email.
    Use antes de qualquer acao sobre o lead."""
    base = {"carlos@datalog.io": {"empresa": "DataLog", "plano": "pro", "mrr": 490}}
    return base.get(email, {"erro": "lead nao encontrado"})

@ferramenta
def calcular_score_lead(plano: str, mrr: int) -> dict:
    """Calcula o score de 0 a 100 de um lead.
    Use DEPOIS de buscar os dados dele."""
    pontos = {"trial": 10, "pro": 40, "enterprise": 70}.get(plano, 0)
    return {"score": pontos + min(mrr // 20, 30)}

# Compare com a Aula 0.3: nenhum schema escrito a mao.
for provider in ["anthropic", "openai"]:
    exe = rodar_agente("Qual o score do lead carlos@datalog.io?",
                      schemas(), executores(), provider=provider)
    print(provider, exe.parou_por, exe.iteracoes, "->", exe.resposta)

```

Resultado esperado: o agente encadeia busca e score exatamente como na aula 0.3, nos dois providers. Compare os dois arquivos lado a lado: sumiram cerca de trinta linhas de schema duplicado, e o comportamento é idêntico. Esse é o ganho.

Passo 5 — Os testes

O que e por quê: dez testes determinísticos. O mais importante deles não testa schema nenhum: é a fixture que isola o estado global.

```

@pytest.fixture(autouse=True)
def registro_limpo():
    """O REGISTRO e um dict de MODULO: ferramentas registradas num
    teste vazariam para o seguinte. Salvamos e restauramos o estado.
    Padrao que vale sempre que houver estado global."""
    original = dict(registro.REGISTRO)
    registro.REGISTRO.clear()
    yield

```

```
registro.REGISTRO.clear()
registro.REGISTRO.update(original)
```

Sem isso, uma ferramenta registrada num teste vazaria para o seguinte, e você teria testes que passam sozinhos e falham na suíte, ou pior, o contrário. **Sempre que houver estado de módulo, haverá uma fixture assim.**

E os dois testes que provam a tese da aula:

```
# ----- 3 e 4. os dois dialetos, do mesmo contrato -----

def test_schema_anthropic():
    _exemplo()
    s = schemas_para("anthropic")[0]
    assert s["name"] == "buscar_lead"
    assert s["input_schema"]["required"] == ["email"]
    assert "function" not in s          # sem envelope na Anthropic

def test_schema_openai():
    _exemplo()
    s = schemas_para("openai")[0]
    assert s["type"] == "function"
    assert s["function"]["name"] == "buscar_lead"
    # O miolo é IDENTICO ao da Anthropic: so a embalagem muda.
    assert s["function"]["parameters"] == \
        schemas_para("anthropic")[0]["input_schema"]
```

A última asserção é a mais importante do arquivo: o miolo gerado para os dois providers é literalmente o mesmo objeto. Só a embalagem difere.

Resultado esperado: `pytest` mostra `22 passed, 2 deselected`, somando os testes das aulas anteriores.

Passo 6 — Commit e tag

```
pytest -v          # esperado: 22 passed, 2 deselected
git status         # .env e scratch/ NAO podem aparecer
git add .
git commit -m "Aula 0.4: registro de ferramentas e schema portavel"
git tag aula-0.4
git push && git push --tags
```

5. Portabilidade

O que você construiu em oitenta linhas é exatamente o que o `@tool` do LangChain e o `@function_tool` do OpenAI Agents SDK fazem por baixo: introspecção da função para gerar o schema. A diferença é que agora você não olha mais para um decorator de framework como mágica.

E na Fase 2, um MCP server expõe precisamente estas mesmas informações (nome, descrição, JSON Schema dos parâmetros) por um transporte padronizado. O seu `Ferramenta` é o mesmo contrato, sem o protocolo em volta.

6. Terminei quando

O critério da aula: **adicionar uma ferramenta nova é escrever uma função, sem tocar no laço.**

- Você acrescenta uma terceira ferramenta ao rascunho e ela aparece nos dois dialetos sem você escrever schema.
- O `agente.py` não foi modificado, e o agente continua encadeando as ferramentas.
- Remova o type hint de um parâmetro: a importação falha na hora, com mensagem clara.
- `pytest` mostra 22 passed, 2 deselected.

7. Erros comuns e troubleshooting

- **TypeError falando de type hint ao importar.** É o comportamento correto. Anote o tipo do parâmetro.
- **Tipo não suportado.** Você usou `list`, `dict` ou um tipo do `typing`. O `MAPA_TIPOS` cobre str, int, float e bool. Ampliar é fácil, mas só quando o projeto precisar.
- **Teste passa sozinho e falha na suíte.** Clássico de estado global. É exatamente o que a fixture `registro_limpo` resolve; confirme que ela está com `autouse=True`.
- **A ferramenta não aparece nos schemas.** O módulo que a define nunca foi importado, então o decorator nunca rodou. Registro por decorator depende de importação.
- **O modelo ignora a ferramenta.** A docstring está vaga. Ela é literalmente o texto que o modelo lê para decidir. Design de descrição é o tema da próxima aula.

8. Ponte para a próxima aula

O registro está pronto, mas as ferramentas ainda são mocks de duas linhas. Na **Aula 0.5, As três ferramentas reais do domínio**, o Núcleo ganha CRM, agenda e tickets sobre SQLite, e o foco muda de mecânica para design: como nome, descrição e parâmetros mudam drasticamente a acurácia de seleção do modelo. Descrição de ferramenta é prompt, e é ali que um agente ganha ou perde qualidade.